INNOPOLIS UNIVERSITY

SNE MASTER
TRACK 2026

COURSE: [S26] Secure Development

Lab 3 Report

On

Fuzzing for Software Security Testing

Student Name: HASIB AL TAHSIN

Application ID: 187623
Student ID: 24

Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

Submission Date: 26.02.2026

Setup:

Installing AFL:

```
hasib@runner-24:~$ sudo apt install afl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Package afl is not available, but is referred to by another package.
This may mean that the package is missing, has been obsoleted, or
is only available from another source
However the following packages replace it:
  afl++

E: Package 'afl' has no installation candidate
```

Since there was no installation candidate for that package, I installed the *afl++* package in the next step.

```
hasib@runner-24:~$ sudo apt install afl++
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  afl++-doc clang-17 libclang-common-17-dev libclang-cpp17t64
  libclang-rt-17-dev libclang1-17t64 libllvm17t64 llvm-17 llvm-17-dev
  llvm-17-linker-tools llvm-17-runtime llvm-17-tools
Suggested packages:
  gnuplot clang-17-doc wasi-libc llvm-17-doc
The following NEW packages will be installed:
  afl++ afl++-doc clang-17 libclang-common-17-dev libclang-cpp17t64
  libclang-rt-17-dev libclang1-17t64 libllvm17t64 llvm-17 llvm-17-dev
  llvm-17-linker-tools llvm-17-runtime llvm-17-tools
0 upgraded, 13 newly installed, 0 to remove and 30 not upgraded.
Need to get 120 MB of archives.
After this operation, 724 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://ru.archive.ubuntu.com/ubuntu noble/main amd64 libllvm17t64 amd64 1:17.0.6-9ubuntu1 [26.2 MB]
Get:2 http://ru.archive.ubuntu.com/ubuntu noble/universe amd64 libclang-cpp17t64 amd64 1:17.0.6-9ubuntu1 [12.9 MB]
Get:3 http://ru.archive.ubuntu.com/ubuntu noble/universe amd64 libclang-common-17-dev amd64 1:17.0.6-9ubuntu1 [690 kB]
Get:4 http://ru.archive.ubuntu.com/ubuntu noble/universe amd64 llvm-17-linker-tools amd64 1:17.0.6-9ubuntu1 [1,308 kB]
Get:5 http://ru.archive.ubuntu.com/ubuntu noble/universe amd64 libclang1-17t64 amd64 1:17.0.6-9ubuntu1 [7,353 kB]
Get:6 http://ru.archive.ubuntu.com/ubuntu noble/universe amd64 clang-17 amd64 1:17.0.6-9ubuntu1 [78.6 kB]
Get:7 http://ru.archive.ubuntu.com/ubuntu noble/universe amd64 afl++ amd64 4.09c-1ubuntu3 [560 kB]
```

Creating a Target Application: Saving the following C code as vulnerable.c:

```
hasib@runner-24:~$ mkdir -p ~/fuzzing-lab/basic
```

```
hasib@runner-24:~$ cd ~/fuzzing-lab/basic
```

Insert *vulnerable.c* file:

```
hasib@runner-24:~/fuzzing-lab/basic$ nano vulnerable.c
```

```
 GNU nano 7.2                    vulnerable.c *
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
void vulnerable_function(char *input) {
char buffer[100];
strcpy(buffer, input); // Potential buffer overflow
}
int main(int argc, char *argv[]) {
if (argc > 1) {
FILE *file = fopen(argv[1], "r");
if (!file) {
perror("Error opening file");
return 1;
}
char input[1024];
size_t bytesRead = fread(input, 1, sizeof(input) - 1,
file);
fclose(file);
input[bytesRead] = '\0';
vulnerable_function(input);
} else {
printf("Usage: %s <filename>\n", argv[0]);
}
return 0;
}
```

Compiling the Target Application with AFL:

Compile:

```
hasib@runner-24:~/fuzzing-lab/basic$ afl-gcc -O0 -g -o vulnerable vulnerable.c
afl-cc++4.09c by Michal Zalewski, Laszlo Szekeres, Marc Heuse - mode: GCC-GCC
[!] WARNING: You are using outdated instrumentation, install LLVM and/or gcc-plugin and
use afl-clang-fast/afl-clang-lto/afl-gcc-fast instead!
afl-as++4.09c by Michal Zalewski
[+] Instrumented 12 locations (64-bit, non-hardened mode, ratio 100%).
```

Verifing executable:

```
hasib@runner-24:~/fuzzing-lab/basic$ ls -la
total 32
drwxrwxr-x 2 hasib hasib  4096 Feb 24 22:05 .
drwxrwxr-x 3 hasib hasib  4096 Feb 24 22:02 ..
-rwxrwxr-x 1 hasib hasib 20368 Feb 24 22:05 vulnerable
-rw-rw-r-- 1 hasib hasib   645 Feb 24 22:04 vulnerable.c
```

Lab Steps

Step 1: Understand the Target Application

➢ Review the *vulnerable.c* code and identify the potential vulnerability (buffer overflow).

In Step 1, I first review the *vulnerable.c* code and see that there is a fixed-size stack buffer of 100 bytes called *char buffer[100];* inside *vulnerable_function().* But right after that, *strcpy(buffer, input);* is used. The main problem here is that *strcpy()* does not check the length of the input, so no matter how big the input is, it will copy the entire input into the buffer. As a result, if the input read from the file is more than 100 bytes, then the buffer overflow occurs and the next part of the stack memory is overwritten. This is the potential vulnerability (stack-based buffer overflow) of this program.

➢ Discuss why this vulnerability is dangerous and how it can be exploited.

This vulnerability is dangerous because a stack overflow not only causes the program to "crash" but also allows the attacker to overwrite important data in the stack, such as the saved return address, function frame data, or other control information. This can change the execution flow of the program. An attacker can use specially crafted input to cause the program to be in a state where **(1)** the program will abort with a stack smashing detected (DoS/Crash), or **(2)** if the environment or protection is weak, the attacker can hijack control and execute arbitrary code. Therefore, the buffer overflow caused by *strcpy()* in this code is very critical from a security perspective.

Running the Fuzzer:

Creating an input directory for AFL:

Creating input folder + seed:

```
hasib@runner-24:~/fuzzing-lab/basic$ mkdir -p input
```

Creating seed:

```
hasib@runner-24:~/fuzzing-lab/basic$ echo "seed" > input/seed.txt
```

Verifying:

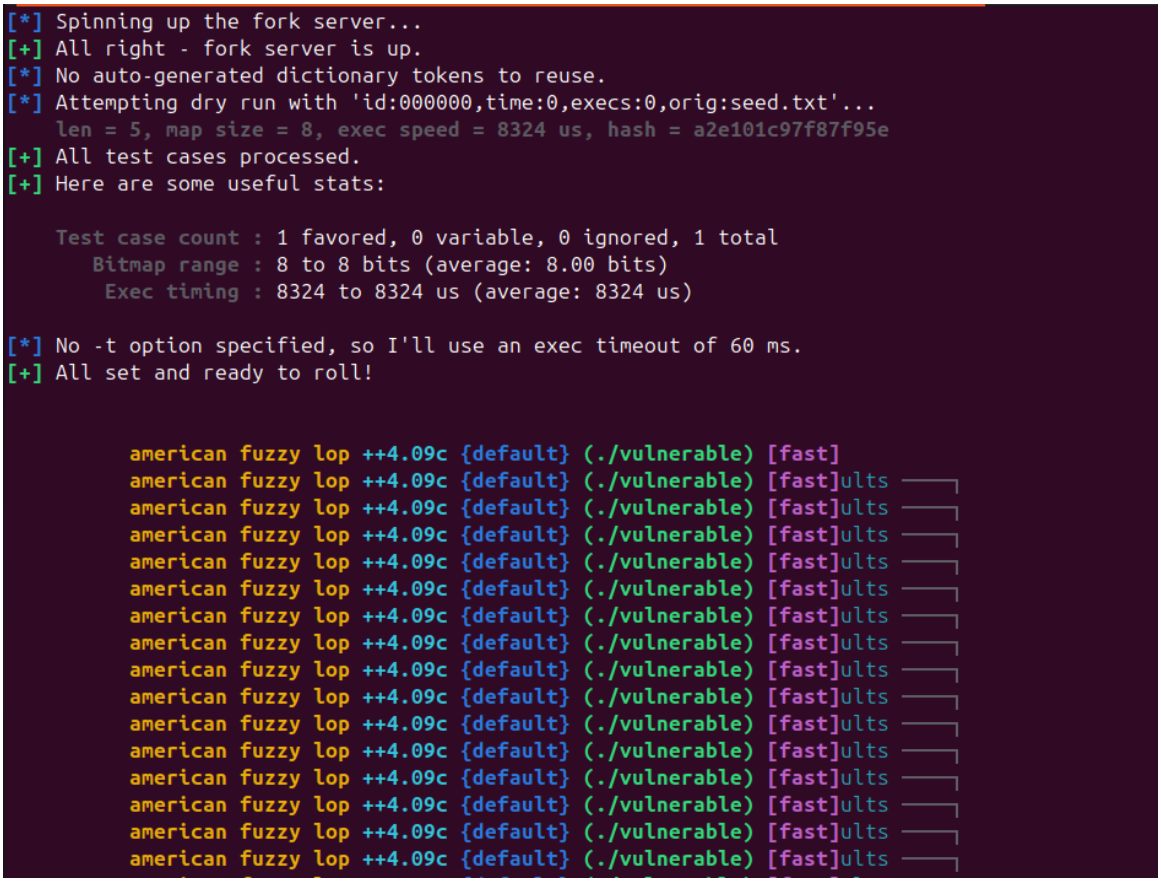
```
hasib@runner-24:~/fuzzing-lab/basic$ ls -la input
total 12
drwxrwxr-x 2 hasib hasib 4096 Feb 24 22:07 .
drwxrwxr-x 3 hasib hasib 4096 Feb 24 22:07 ..
-rw-rw-r-- 1 hasib hasib    5 Feb 24 22:07 seed.txt
```

Running AFL on the target application:

- *-i* input: Specifies the input directory with seed files.
- *-o* output: Specifies the output directory for fuzzing results.
- *./vulnerable @@:* The target application, where @@ is a placeholder for the input file.

```
hasib@runner-24:~/fuzzing-lab/basic$ afl-fuzz -i input -o output -- ./vulnerable @@
afl-fuzz++4.09c based on afl by Michal Zalewski and a large online community
[+] AFL++ is maintained by Marc "van Hauser" Heuse, Dominik Maier, Andrea Fioraldi and Heiko "hexcoder" Eißfeldt
[+] AFL++ is open source, get it at https://github.com/AFLplusplus/AFLplusplus
[+] NOTE: AFL++ >= v3 has changed defaults and behaviours - see README.md
[+] No -M/-S set, autoconfiguring for "-S default"
[*] Getting to work...
[+] Using exponential power schedule (FAST)
[+] Enabled testcache with 50 MB
[+] Generating fuzz data with a length of min=1 max=1048576
[*] Checking core_pattern...
[!] WARNING: Could not check CPU scaling governor
[+] You have 2 CPU cores and 3 runnable tasks (utilization: 150%).
[*] Setting up output directories...
[*] Checking CPU core loadout...
[+] Found a free CPU core, try binding to #0.
[*] Scanning 'input'...
[+] Loaded a total of 1 seeds.
[*] Creating hard links for all input files...
[*] Validating target binary...
[*] Spinning up the fork server...
[+] All right - fork server is up.
[*] No auto-generated dictionary tokens to reuse.
[*] Attempting dry run with 'id:000000,time:0,execs:0,orig:seed.txt'...
    len = 5, map size = 8, exec speed = 8324 us, hash = a2e101c97f87f95e
[+] All test cases processed.
[+] Here are some useful stats:

    Test case count : 1 favored, 0 variable, 0 ignored, 1 total
       Bitmap range : 8 to 8 bits (average: 8.00 bits)
        Exec timing : 8324 to 8324 us (average: 8324 us)
```





Then, pressing *Ctrl + C* turns off the running AFL in the application.

Step 3: Monitor the Fuzzing Process

Observe the AFL interface, which provides real-time statistics such as:

➢ Total Executions: Number of test cases executed.

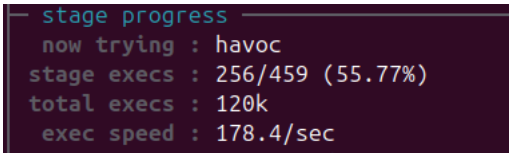
➢ Unique Crashes: Number of unique crashes found.

➢ Hangs: Number of timeouts or hangs.

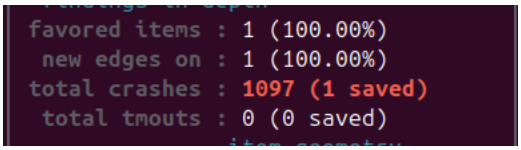
➢ Let the fuzzer run for 5-10 minutes.

After running the *AFL++ fuzzer*, I monitor its interactive UI/dashboard, as this is where I can see the real-time status during fuzzing. After running *afl-fuzz*, I observed a few important things on the AFL screen:

- Total Executions (Number of test cases executed): Shows how many inputs AFL has executed. Your run showed about total execs: 120k (meaning *~1.2 lakh* test cases executed). The higher this number, the more input programs are being tried. The stage execution was *55.77%.* Also, I gave this VM two core CPUs to run this VM, which showed when running the application.

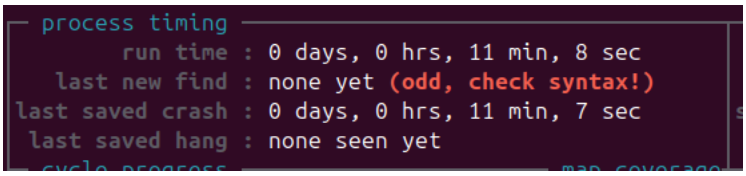


- Unique Crashes (Number of unique crashes found): AFL crashes can occur many times, but a "*saved crash*" means that AFL has saved at least one distinct crash sample. The UI shows saved crashes: 1 (meaning 1 crash file has been saved), although *total crashes: 1097* was shown, meaning that the crash occurred many times, but 1 was saved as unique.



- Hangs (Number of timeouts or hangs): If the program does not finish within a certain time (*timeout*), it is considered a hang. According to the output, saved *hangs: 0* and *total tmouts: 0* were found which means no hang/timeout was found during fuzzing.

At this stage, I let the fuzzer run for about 5-10 minutes (my run time shows that it runs for about 11 minutes: *run time : 0 days, 0 hrs, 11 min, 8 second*).



Then, when enough execution and crash evidence was available, I stopped *fuzzing* by pressing *Ctrl + C* and analyzed the crash artifacts from the output folder in step 1.

Step 4: Analyze the Results

After stopping the *fuzzer*, I checked the output directory:

I looked for files that caused crashes and the one that crashed was *id:000000,sig:06*, which was in *repo 8* of havoc operation. I also found *README.txt* in the crashes folder.

```
hasib@runner-24:~/fuzzing-lab/basic$ ls -la output/default/crashes
total 16
drwx------ 2 hasib hasib 4096 Feb 24 22:21 .
drwx------ 6 hasib hasib 4096 Feb 24 22:32 ..
-rw------- 1 hasib hasib  114 Feb 24 22:21 id:000000,sig:06,src:000000,time:550,execs:98,op:havoc,rep:8
-rw------- 1 hasib hasib  539 Feb 24 22:21 README.txt
```

Then, I found the file *id:000000 seed.txt* in the queue folder.

```
hasib@runner-24:~/fuzzing-lab/basic$ ls -la output/default/queue | head
total 16
drwx------ 3 hasib hasib 4096 Feb 24 22:21 .
drwx------ 6 hasib hasib 4096 Feb 24 22:32 ..
-rw------- 1 hasib hasib    4 Feb 24 22:21 id:000000,time:0,execs:0,orig:seed.txt
drwx------ 6 hasib hasib 4096 Feb 24 22:21 .state
```

Since, there was no hang, no data or files were found in the hangs folder.

```
hasib@runner-24:~/fuzzing-lab/basic$ ls -la output/default/hangs
total 8
drwx------ 2 hasib hasib 4096 Feb 24 22:21 .
drwx------ 6 hasib hasib 4096 Feb 24 22:32 ..
```

Replay the crash:

The real cause of the crash at this stage was found by replaying the crash file: ****stack smashing detected***:* terminated and then *aborted*.

```
hasib@runner-24:~/fuzzing-lab/basic$ ./vulnerable output/default/crashes/id:000000,sig:06*
*** stack smashing detected ***: terminated
Aborted
```

This ASAN output shows that a stack-based buffer overflow occurred in the *vulnerable.c* program. The crash input generated by the *fuzzer* exceeded the limit while being copied to *char buffer[100]* inside *vulnerable_function().* Specifically, the *strcpy()* function copied the data without checking the length of the input, resulting in a *WRITE* of size *115* in a 100-byte buffer. ASAN clearly shows that the overflow occurred at line *vulnerable.c:7,* i.e. during the *strcpy()* call, and the memory access buffer exceeded the specified limit and reached the *redzone* part of the stack. Therefore, the program aborted for safety reasons. In short, this output confirms that the root cause of the crash was the stack buffer overflow caused by *strcpy().*

```
hasib@runner-24:~/fuzzing-lab/basic$ ./vulnerable_asan output/default/crashes/id:000000,sig:06*
=================================================================
==127119==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x753866700094 at pc 0x753868aa7923 bp 0x7fff4e246ad0 sp 0x7fff4e246278
WRITE of size 115 at 0x753866700094 thread T0
    #0 0x753868aa7922 in strcpy ../../../../src/libsanitizer/asan/asan_interceptors.cpp:563
    #1 0x57ee1d82c3ee in vulnerable_function /home/hasib/fuzzing-lab/basic/vulnerable.c:7
    #2 0x57ee1d82c638 in main /home/hasib/fuzzing-lab/basic/vulnerable.c:21
    #3 0x75386862a1c9 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
    #4 0x75386862a28a in __libc_start_main_impl ../csu/libc-start.c:360
    #5 0x57ee1d82c264 in _start (/home/hasib/fuzzing-lab/basic/vulnerable_asan+0x1264) (BuildId: 98baa61671209bcfb1fcc28c9aa52ca1d052333d)

Address 0x753866700094 is located in stack of thread T0 at offset 148 in frame
    #0 0x57ee1d82c338 in vulnerable_function /home/hasib/fuzzing-lab/basic/vulnerable.c:5

  This frame has 1 object(s):
    [48, 148) 'buffer' (line 6) <== Memory access at offset 148 overflows this variable
HINT: this may be a false positive if your program uses some custom stack unwind mechanism, swapcontext or vfork
      (longjmp and C++ exceptions *are* supported)
SUMMARY: AddressSanitizer: stack-buffer-overflow ../../../../src/libsanitizer/asan/asan_interceptors.cpp:563 in strcpy
Shadow bytes around the buggy address:
  0x7538666ffe00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x7538666ffe80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x7538666fff00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x7538666fff80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x753866700000: f1 f1 f1 f1 f1 f1 00 00 00 00 00 00 00 00 00 00
=>0x753866700080: 00 00[04]f3 f3 f3 f3 f3 00 00 00 00 00 00 00 00
  0x753866700100: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x753866700180: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x753866700200: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x753866700280: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x753866700300: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

```
Shadow byte legend (one shadow byte represents 8 application bytes):
  Addressable:           00
  Partially addressable: 01 02 03 04 05 06 07
  Heap left redzone:       fa
  Freed heap region:       fd
  Stack left redzone:      f1
  Stack mid redzone:       f2
  Stack right redzone:     f3
  Stack after return:      f5
  Stack use after scope:   f8
  Global redzone:          f9
  Global init order:       f6
  Poisoned by user:        f7
  Container overflow:      fc
  Array cookie:            ac
  Intra object redzone:    bb
  ASan internal:           fe
  Left alloca redzone:     ca
  Right alloca redzone:    cb
==127119==ABORTING
```

➢ **Looking for files that caused crashes.**

After stopping fuzzing, I first check the *AFL++* result or output directory to see if there is a crash-causing input file. In this case, the result is in output/default/ according to the *AFL++* default structure, so I try to observe what is there in output/default/crashes. The output shows that a crash file has been created inside the output/default/crashes folder:

id:000000,sig:06,src:000000,time:550,execs:98,op:havoc,rep:8 and *README.txt*. This *id:000000,sig:06* file is the specific input sample that crashed the target program and *AFL++* saved it as evidence. Also, if you check output/default/queue, you will see *id:000000,...,orig:seed.txt,* which indicates that the fuzzing started from seed input seed.txt, and the output/default/hangs folder is empty, which means that no hangs/timeouts were saved.

➢ **Observe the crash and discuss why it occurred.**

The crash occurred because the *fuzzer* created a large input, which caused a buffer overflow with *strcpy(),* causing memory corruption and the program aborted/crashed. I understand this by replaying the crash file to confirm the crash. Running the command *./vulnerable output/default/crashes/id:000000,sig:06** gave the output *** *stack smashing detected* ***:* terminated and *Aborted*. The crash occurred because the input created by the *fuzzer* exceeded the char *buffer[100]* limit inside *vulnerable_function(),* and the code used *strcpy(buffer, input)* which copies the input without checking the length. As a result, the buffer overflowed while copying the large input and overwrote the stack memory, resulting in the program aborting. The ASAN output also clearly shows that *AddressSanitizer: stack-buffer-overflow, WRITE* of size *115*, and buffer is seen as the overflowed variable, which confirms that the main cause of the crash is a stack-based buffer overflow with *strcpy()*.

Step 5: Fix the Vulnerability

➢ Modify the vulnerable.c code to fix the buffer overflow (e.g., use strncpy instead of strcpy).

➢ Recompile and rerun the fuzzer to verify the fix.

Fixing code

edit:

When modifying the *vulnerable.c* file, the previous unsafe strcpy(buffer, input); line was removed and replaced with *strncpy(buffer, input, sizeof(buffer) - 1);* so that the maximum buffer size is not exceeded during copying. *Buffer[sizeof(buffer) - 1] = '\0';* was also added so that the string is properly null-terminated. Here, by using *sizeof(buffer) - 1*, one byte is intentionally left empty so that overflow does not occur and the program can handle input safely.

```
hasib@runner-24:~/fuzzing-lab/basic$ nano vulnerable.c
```

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void vulnerable_function(char *input) {
    char buffer[100];
    strncpy(buffer, input, sizeof(buffer)-1);
    buffer[sizeof(buffer)-1] = '\0'; // Potential buffer overflow
}

int main(int argc, char *argv[]) {
    if (argc > 1) {
        FILE *file = fopen(argv[1], "r");
        if (!file) {
            perror("Error opening file");
            return 1;
        }
        char input[1024];
        size_t bytesRead = fread(input, 1, sizeof(input) - 1, file);
        fclose(file);
        input[bytesRead] = '\0';
        vulnerable_function(input);
    } else {
        printf("Usage: %s <filename>\n", argv[0]);
    }
    return 0;
}
```

I replaced it

Recompile:

```
hasib@runner-24:~/fuzzing-lab/basic$ afl-gcc -o vulnerable vulnerable.c
afl-cc++4.09c by Michal Zalewski, Laszlo Szekeres, Marc Heuse - mode: GCC-GCC
 [!] WARNING: You are using outdated instrumentation, install LLVM and/or gcc-plugin and use afl-clang-fast/afl-clang-lto/afl-gcc-fast instead!
afl-as++4.09c by Michal Zalewski
 [+] Instrumented 9 locations (64-bit, non-hardened mode, ratio 100%).
```

Cleaning old output:

```
hasib@runner-24:~/fuzzing-lab/basic$ rm -rf output
```

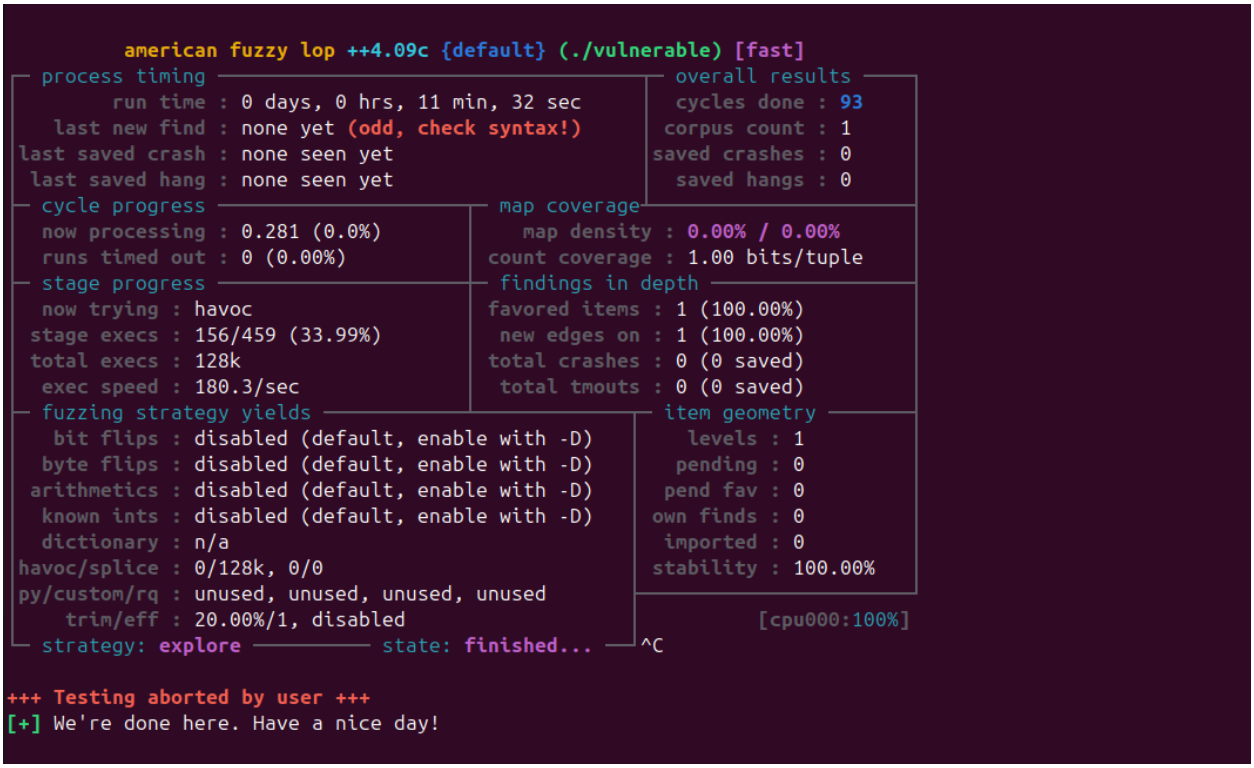
Running *fuzzer* again:

```
hasib@runner-24:~/fuzzing-lab/basic$ afl-fuzz -i input -o output -- ./vulnerable @@
afl-fuzz++4.09c based on afl by Michal Zalewski and a large online community
[+] AFL++ is maintained by Marc "van Hauser" Heuse, Dominik Maier, Andrea Fioraldi and Heiko "hexcoder" Eißfeldt
[+] AFL++ is open source, get it at https://github.com/AFLplusplus/AFLplusplus
[+] NOTE: AFL++ >= v3 has changed defaults and behaviours - see README.md
[+] No -M/-S set, autoconfiguring for "-S default"
[*] Getting to work...
[+] Using exponential power schedule (FAST)
[+] Enabled testcache with 50 MB
[+] Generating fuzz data with a length of min=1 max=1048576
[*] Checking core_pattern...
[!] WARNING: Could not check CPU scaling governor
[+] You have 2 CPU cores and 2 runnable tasks (utilization: 100%).
[*] Setting up output directories...
[*] Checking CPU core loadout...
[+] Found a free CPU core, try binding to #0.
[*] Scanning 'input'...
[+] Loaded a total of 1 seeds.
[*] Creating hard links for all input files...
[*] Validating target binary...
[*] Spinning up the fork server...
[+] All right - fork server is up.
[*] No auto-generated dictionary tokens to reuse.
[*] Attempting dry run with 'id:000000,time:0,execs:0,orig:seed.txt'...
    len = 5, map size = 6, exec speed = 8847 us, hash = f521c56d100066c8
[+] All test cases processed.
[+] Here are some useful stats:
```

```
[+] You have 2 CPU cores and 2 runnable tasks (utilization: 100%).
[*] Setting up output directories...
[*] Checking CPU core loadout...
[+] Found a free CPU core, try binding to #0.
[*] Scanning 'input'...
[+] Loaded a total of 1 seeds.
[*] Creating hard links for all input files...
[*] Validating target binary...
[*] Spinning up the fork server...
[+] All right - fork server is up.
[*] No auto-generated dictionary tokens to reuse.
[*] Attempting dry run with 'id:000000,time:0,execs:0,orig:seed.txt'...
    len = 5, map size = 6, exec speed = 8847 us, hash = f521c56d100066c8
[+] All test cases processed.
[+] Here are some useful stats:

    Test case count : 1 favored, 0 variable, 0 ignored, 1 total
       Bitmap range : 6 to 6 bits (average: 6.00 bits)
        Exec timing : 8847 to 8847 us (average: 8847 us)

[*] No -t option specified, so I'll use an exec timeout of 60 ms.
[+] All set and ready to roll!
```



Verifying crashes:

No crash file or *README.txt* was found in the Crashes folder.

```
hasib@runner-24:~/fuzzing-lab/basic$ ls -la output/default/crashes
total 8
drwx------ 2 hasib hasib 4096 Feb 24 22:43 .
drwx------ 6 hasib hasib 4096 Feb 24 22:55 ..
```

```
hasib@runner-24:~/fuzzing-lab/basic$ echo "Crashes after fix:"; ls -1 output/default/crashes | wc -l
Crashes after fix:
0
```

In *AFL++,* there are no crashes or 0 in *output/default/crashes* compared to before.

Questions

1. What is the purpose of fuzzing in software security testing?

Answer:

The main purpose of fuzzing is to automatically test software with random, malformed, or mutated inputs to detect unexpected behavior, crashes, memory corruption, or security vulnerabilities. It is a dynamic testing technique in which a program is run in a real execution

environment and is stress tested with input. If an input causes the program to crash or behave abnormally, it indicates a potential security flaw. As seen in this lab, fuzzing has been used to quickly detect *stack-based buffer overflows*.

2. How does AFL generate test cases to find vulnerabilities?

Answer:

AFL (*American Fuzzy Lop*) first starts working with one or more seed inputs. Then it compiles the target program with instrumentation, so that it can monitor which code paths are being executed during execution (coverage-based feedback). AFL uses this coverage information to mutate the seed inputs in various ways. Such as bit flip, byte flip, arithmetic mutation, havoc stage, splice, etc. When a mutated input explores a new code path, AFL gives it importance and adds it to the corpus. If an input causes a crash or hang, AFL saves it separately. Through this feedback-driven mutation approach, AFL gradually penetrates deep into the program and finds vulnerabilities.

3. What other types of vulnerabilities can fuzzing detect besides buffer overflows?
Answer:

In addition to *buffer overflow*, fuzzing can detect many other types of vulnerabilities. For example: *out-of-bounds read/write*, *use-after-free, null pointer dereference, integer overflow/underflow, format string vulnerability, assertion failure, infinite loop or hang, parsing error, memory leak, and logic-based crash*. Especially when fuzzing is used with *AddressSanitizer (*ASAN), problems related to memory safety and undefined behavior can be detected more easily.

The vulnerability that we detected in this lab was stack-based *buffer overflow*. However, the fuzzing technique itself is not limited to buffer overflow. I saw in the *AFL++* dashboard that it tracks crash, hang, and timeout separately (*saved crashes, saved hangs, total execs, havoc, total tmouts*), which suggests that fuzzing is capable of detecting many other types of vulnerabilities.

As seen in the ASAN *replay output, AddressSanitizer: stack-buffer-overflow* and *memory write violation* were detected, similarly, using ASAN or UBSAN, fuzzing can also clearly report other memory safety issues such as use-after-free, heap overflow, null dereference, etc.

4. How can you improve the efficiency of a fuzzing campaign?

Answer:

Several techniques can be used to make a fuzzing campaign more effective.

a. Using a good and diverse seed corpus allows AFL to explore new code paths faster.

b. Compiling the target program with advanced instrumentation (such as *afl-clang-fast* or *LTO mode*) gives better results.

c. Using sanitizers (ASAN) makes vulnerability detection more accurate.

d. Using parallel fuzzing *(-M / -S mode)* allows fuzzing to be run on multiple cores simultaneously.

e. Optimizing execution timeout, disabling unnecessary functionality, and running fuzzing for a long time increases performance.

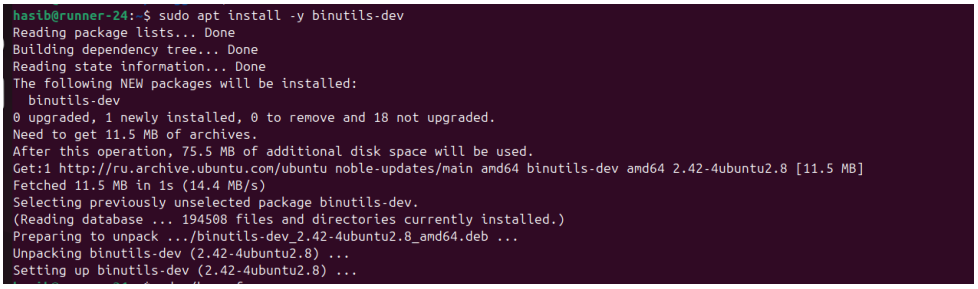
Bonus Challenge

1. **Use a different fuzzing tool (e.g., *libFuzzer* or *Honggfuzz*) and compare its effectiveness with AFL.**
2. **Write a custom *fuzzer* in Python using the random or hypothesis library to test the same application.**

Bonus Challenge

1.

Installing the required packages:



git clone:

```
hasib@runner-24:~$ git clone https://github.com/google/honggfuzz.git
Cloning into 'honggfuzz'...
remote: Enumerating objects: 61861, done.
remote: Counting objects: 100% (521/521), done.
remote: Compressing objects: 100% (142/142), done.
remote: Total 61861 (delta 448), reused 379 (delta 379), pack-reused 61340 (from 3)
Receiving objects: 100% (61861/61861), 187.75 MiB | 15.00 MiB/s, done.
Resolving deltas: 100% (33187/33187), done.
Updating files: 100% (13047/13047), done.
```

Rebuild (clean):

Going to *Honggfuzz* folder:

```
hasib@runner-24:~$ cd ~/honggfuzz
```

Make clean:

```
hasib@runner-24:~/honggfuzz$ make clean
rm -f -r core Makefile.bak cmdline.o dict.o display.o fuzz.o honggfuzz.o input.o mangle.o power.o report.o sanitizers.o socketfuzzer.o subproc.o linux/arch.o linux
/bfd.o linux/perf.o linux/pt.o linux/trace.o linux/unwind.o honggfuzz hfuzz_cc/hfuzz-cc libhfuzz/libhfuzz.a libhfuzz/libhfuzz.so libhfuzz/fetch.o libhfuzz/instrume
nt.o libhfuzz/linux.o libhfuzz/memorycmp.o libhfuzz/performance.o libhfuzz/persistent.o libhfcommon/libhfcommon.a libhfcommon/files.o libhfcommon/log.o libhfcommon
/ns.o libhfcommon/util.o libhfnetdriver/libhfnetdriver.a libhfnetdriver/netdriver.o  obj libs ./*.o ./*~ ./core ./*.a ./*.dSYM ./*.la ./*.so ./*.dylib linux/*.o li
nux/*~ linux/core linux/*.a linux/*.dSYM linux/*.la linux/*.so linux/*.dylib mac/*.o mac/*~ mac/core mac/*.a mac/*.dSYM mac/*.la mac/*.so mac/*.dylib netbsd/*.o ne
tbsd/*~ netbsd/core netbsd/*.a netbsd/*.dSYM netbsd/*.la netbsd/*.so netbsd/*.dylib posix/*.o posix/*~ posix/core posix/*.a posix/*.dSYM posix/*.la posix/*.so posi
x/*.dylib libhfuzz/*.o libhfuzz/*~ libhfuzz/core libhfuzz/*.a libhfuzz/*.dSYM libhfuzz/*.la libhfuzz/*.so libhfuzz/*.dylib libhfcommon/*.o libhfcommon/*~ libhfcomm
on/core libhfcommon/*.a libhfcommon/*.dSYM libhfcommon/*.la libhfcommon/*.so libhfcommon/*.dylib libhfnetdriver/*.o libhfnetdriver/*~ libhfnetdriver/core libhfnetd
river/*.a libhfnetdriver/*.dSYM libhfnetdriver/*.la libhfnetdriver/*.so libhfnetdriver/*.dylib
```

Make:

```
hasib@runner-24:~/honggfuzz$ make
cc -c -O3 -mtune=native -funroll-loops -std=c11 -I/usr/local/include -D_GNU_SOURCE -Wall -Wextra -Werror -Wno-format-truncation -Wno-override-init -I. -D_FILE_OFFS
ET_BITS=64 -finline-limit=4000 -D_HF_ARCH_LINUX -D_HF_ARCH_LINUX  -o cmdline.o cmdline.c
cc -c -O3 -mtune=native -funroll-loops -std=c11 -I/usr/local/include -D_GNU_SOURCE -Wall -Wextra -Werror -Wno-format-truncation -Wno-override-init -I. -D_FILE_OFFS
ET_BITS=64 -finline-limit=4000 -D_HF_ARCH_LINUX -D_HF_ARCH_LINUX  -o dict.o dict.c
cc -c -O3 -mtune=native -funroll-loops -std=c11 -I/usr/local/include -D_GNU_SOURCE -Wall -Wextra -Werror -Wno-format-truncation -Wno-override-init -I. -D_FILE_OFFS
ET_BITS=64 -finline-limit=4000 -D_HF_ARCH_LINUX -D_HF_ARCH_LINUX  -o display.o display.c
cc -c -O3 -mtune=native -funroll-loops -std=c11 -I/usr/local/include -D_GNU_SOURCE -Wall -Wextra -Werror -Wno-format-truncation -Wno-override-init -I. -D_FILE_OFFS
ET_BITS=64 -finline-limit=4000 -D_HF_ARCH_LINUX -D_HF_ARCH_LINUX  -o fuzz.o fuzz.c
cc -c -O3 -mtune=native -funroll-loops -std=c11 -I/usr/local/include -D_GNU_SOURCE -Wall -Wextra -Werror -Wno-format-truncation -Wno-override-init -I. -D_FILE_OFFS
ET_BITS=64 -finline-limit=4000 -D_HF_ARCH_LINUX -D_HF_ARCH_LINUX  -o honggfuzz.o honggfuzz.c
cc -c -O3 -mtune=native -funroll-loops -std=c11 -I/usr/local/include -D_GNU_SOURCE -Wall -Wextra -Werror -Wno-format-truncation -Wno-override-init -I. -D_FILE_OFFS
ET_BITS=64 -finline-limit=4000 -D_HF_ARCH_LINUX -D_HF_ARCH_LINUX  -o input.o input.c
cc -c -O3 -mtune=native -funroll-loops -std=c11 -I/usr/local/include -D_GNU_SOURCE -Wall -Wextra -Werror -Wno-format-truncation -Wno-override-init -I. -D_FILE_OFFS
ET_BITS=64 -finline-limit=4000 -D_HF_ARCH_LINUX -D_HF_ARCH_LINUX  -o mangle.o mangle.c
cc -c -O3 -mtune=native -funroll-loops -std=c11 -I/usr/local/include -D_GNU_SOURCE -Wall -Wextra -Werror -Wno-format-truncation -Wno-override-init -I. -D_FILE_OFFS
ET_BITS=64 -finline-limit=4000 -D_HF_ARCH_LINUX -D_HF_ARCH_LINUX  -o power.o power.c
cc -c -O3 -mtune=native -funroll-loops -std=c11 -I/usr/local/include -D_GNU_SOURCE -Wall -Wextra -Werror -Wno-format-truncation -Wno-override-init -I. -D_FILE_OFFS
ET_BITS=64 -finline-limit=4000 -D_HF_ARCH_LINUX -D_HF_ARCH_LINUX  -o report.o report.c
cc -c -O3 -mtune=native -funroll-loops -std=c11 -I/usr/local/include -D_GNU_SOURCE -Wall -Wextra -Werror -Wno-format-truncation -Wno-override-init -I. -D_FILE_OFFS
ET_BITS=64 -finline-limit=4000 -D_HF_ARCH_LINUX -D_HF_ARCH_LINUX  -o sanitizers.o sanitizers.c
cc -c -O3 -mtune=native -funroll-loops -std=c11 -I/usr/local/include -D_GNU_SOURCE -Wall -Wextra -Werror -Wno-format-truncation -Wno-override-init -I. -D_FILE_OFFS
ET_BITS=64 -finline-limit=4000 -D_HF_ARCH_LINUX -D_HF_ARCH_LINUX  -o socketfuzzer.o socketfuzzer.c
cc -c -O3 -mtune=native -funroll-loops -std=c11 -I/usr/local/include -D_GNU_SOURCE -Wall -Wextra -Werror -Wno-format-truncation -Wno-override-init -I. -D_FILE_OFFS
ET_BITS=64 -finline-limit=4000 -D_HF_ARCH_LINUX -D_HF_ARCH_LINUX  -o subproc.o subproc.c
cc -c -O3 -mtune=native -funroll-loops -std=c11 -I/usr/local/include -D_GNU_SOURCE -Wall -Wextra -Werror -Wno-format-truncation -Wno-override-init -I. -D_FILE_OFFS
ET_BITS=64 -finline-limit=4000 -D_HF_ARCH_LINUX -D_HF_ARCH_LINUX  -o linux/arch.o linux/arch.c
cc -c -O3 -mtune=native -funroll-loops -std=c11 -I/usr/local/include -D_GNU_SOURCE -Wall -Wextra -Werror -Wno-format-truncation -Wno-override-init -I. -D_FILE_OFFS
ET_BITS=64 -finline-limit=4000 -D_HF_ARCH_LINUX -D_HF_ARCH_LINUX  -o linux/bfd.o linux/bfd.c
cc -c -O3 -mtune=native -funroll-loops -std=c11 -I/usr/local/include -D_GNU_SOURCE -Wall -Wextra -Werror -Wno-format-truncation -Wno-override-init -I. -D_FILE_OFFS
ET_BITS=64 -finline-limit=4000 -D_HF_ARCH_LINUX -D_HF_ARCH_LINUX  -o linux/perf.o linux/perf.c
cc -c -O3 -mtune=native -funroll-loops -std=c11 -I/usr/local/include -D_GNU_SOURCE -Wall -Wextra -Werror -Wno-format-truncation -Wno-override-init -I. -D_FILE_OFFS
```

```
cc -c -O3 -mtune=native -funroll-loops -std=c11 -I/usr/local/include -D_GNU_SOURCE -Wall -Wextra -Werror -Wno-format-truncation -Wno-override-init -I. -D_FILE_OFFS
ET_BITS=64 -finline-limit=4000 -D_HF_ARCH_LINUX -D_HF_ARCH_LINUX -fPIC -fno-stack-protector -U_FORTIFY_SOURCE -D_FORTIFY_SOURCE=0   -o libhfuzz/memorycmp.o libhfuz
z/memorycmp.c
cc -c -O3 -mtune=native -funroll-loops -std=c11 -I/usr/local/include -D_GNU_SOURCE -Wall -Wextra -Werror -Wno-format-truncation -Wno-override-init -I. -D_FILE_OFFS
ET_BITS=64 -finline-limit=4000 -D_HF_ARCH_LINUX -D_HF_ARCH_LINUX -fPIC -fno-stack-protector -U_FORTIFY_SOURCE -D_FORTIFY_SOURCE=0   -o libhfuzz/performance.o libhf
uzz/performance.c
cc -c -O3 -mtune=native -funroll-loops -std=c11 -I/usr/local/include -D_GNU_SOURCE -Wall -Wextra -Werror -Wno-format-truncation -Wno-override-init -I. -D_FILE_OFFS
ET_BITS=64 -finline-limit=4000 -D_HF_ARCH_LINUX -D_HF_ARCH_LINUX -fPIC -fno-stack-protector -U_FORTIFY_SOURCE -D_FORTIFY_SOURCE=0   -o libhfuzz/persistent.o libhfu
zz/persistent.c
ar rcs libhfuzz/libhfuzz.a libhfuzz/fetch.o libhfuzz/instrument.o libhfuzz/linux.o libhfuzz/memorycmp.o libhfuzz/performance.o libhfuzz/persistent.o
cc -c -O3 -mtune=native -funroll-loops -std=c11 -I/usr/local/include -D_GNU_SOURCE -Wall -Wextra -Werror -Wno-format-truncation -Wno-override-init -I. -D_FILE_OFFS
ET_BITS=64 -finline-limit=4000 -D_HF_ARCH_LINUX -D_HF_ARCH_LINUX -fPIC -fno-stack-protector -U_FORTIFY_SOURCE -D_FORTIFY_SOURCE=0   -o libhfnetdriver/netdriver.o l
ibhfnetdriver/netdriver.c
ar rcs libhfnetdriver/libhfnetdriver.a libhfnetdriver/netdriver.o
cc -o hfuzz_cc/hfuzz-cc hfuzz_cc/hfuzz-cc.c libhfcommon/libhfcommon.a -pthread -L/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/lib -lm -L/usr/local/inclu
de -lunwind-ptrace -lunwind-generic -lunwind  -llzma -lopcodes -lbfd -lrt -ldl -lm -latomic -O3 -mtune=native -funroll-loops -std=c11 -I/usr/local/include -D_GNU_S
OURCE -Wall -Wextra -Werror -Wno-format-truncation -Wno-override-init -I. -D_FILE_OFFSET_BITS=64 -finline-limit=4000 -D_HF_ARCH_LINUX -D_HF_ARCH_LINUX  -D_HFUZZ_IN
C_PATH=/home/hasib/honggfuzz
cc -shared libhfuzz/fetch.o libhfuzz/instrument.o libhfuzz/linux.o libhfuzz/memorycmp.o libhfuzz/performance.o libhfuzz/persistent.o libhfcommon/files.o libhfcommo
n/log.o libhfcommon/ns.o libhfcommon/util.o -pthread -L/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/lib -lm -L/usr/local/include -lunwind-ptrace -lunwin
d-generic -lunwind  -llzma -lopcodes -lbfd -lrt -ldl -lm -latomic -o libhfuzz/libhfuzz.so
```

system-wide install:

```
hasib@runner-24:~/honggfuzz$ sudo make install
mkdir -p -m 755 ${DESTDIR}/usr/local/bin
install -m 755 honggfuzz ${DESTDIR}/usr/local/bin
install -m 755 hfuzz_cc/hfuzz-cc ${DESTDIR}/usr/local/bin
install -m 755 hfuzz_cc/hfuzz-clang ${DESTDIR}/usr/local/bin
install -m 755 hfuzz_cc/hfuzz-clang++ ${DESTDIR}/usr/local/bin
install -m 755 hfuzz_cc/hfuzz-gcc ${DESTDIR}/usr/local/bin
install -m 755 hfuzz_cc/hfuzz-g++ ${DESTDIR}/usr/local/bin
install -d ${DESTDIR}/usr/local/include/libhfcommon
install -d ${DESTDIR}/usr/local/include/libhfuzz
install -d ${DESTDIR}/usr/local/include/libhnetdriver
install -m 755 includes/libhfcommon/*.h ${DESTDIR}/usr/local/include/libhfcommon
install -m 755 includes/libhfuzz/*.h ${DESTDIR}/usr/local/include/libhfuzz
install -m 755 includes/libhfnetdriver/*.h ${DESTDIR}/usr/local/include/libhnetdriver
```

Checking help for build success:

```
hasib@runner-24:~/honggfuzz$ ./honggfuzz --help | head
Usage: ./honggfuzz [options] -- path_to_command [args]
Options:
 --help|-h
       Help plz..
 --input|-i VALUE
       Path to a directory containing initial file corpus
 --output|-o VALUE
       Output data (new dynamic coverage corpus, or the minimized coverage corpus) is written to this directory (default: input directory is re-used)
 --persistent|-P
       Enable persistent fuzzing (use hfuzz_cc/hfuzz-clang to compile code). This will be auto-detected!!!
```

Compiling (with ASAN) *vulnerable.c*:

Going to a specific folder:

```
hasib@runner-24:~/honggfuzz$ cd ~/fuzzing-lab/basic
```

The code given first in the lab instructions:

```
hasib@runner-24:~/fuzzing-lab/basic$ nano vulnerable.c
```

```
 GNU nano 7.2                                                vulnerable.c *
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
void vulnerable_function(char *input) {
char buffer[100];
strcpy(buffer, input); // Potential buffer overflow
}
int main(int argc, char *argv[]) {
if (argc > 1) {
FILE *file = fopen(argv[1], "r");
if (!file) {
perror("Error opening file");
return 1;
}
char input[1024];
size_t bytesRead = fread(input, 1, sizeof(input) - 1,
file);
fclose(file);
input[bytesRead] = '\0';
vulnerable_function(input);
} else {
printf("Usage: %s <filename>\n", argv[0]);
}
return 0;
}
```

Folder Creation:

```
hasib@runner-24:~/fuzzing-lab/basic$ mkdir -p input_hf
```

Creating a seed:

```
hasib@runner-24:~/fuzzing-lab/basic$ echo "seed" > input_hf/seed.txt
```

***Honggfuzz* Run:**

After running *Honggfuzz*, the output shows that a total of *6,474 iterations* were completed in about *7 minutes and 23 seconds*, and the tool worked in *Feedback-Driven Mode*. Although a total of *275 crashes* were detected, there was *1 unique crash,* meaning the same vulnerability was triggered repeatedly. In addition, *189 timeouts* were found (with a 1 second timeout setting), indicating that some input caused the program to behave in a delayed or unusual way. The corpus size increased from *1 seed to 11 inputs*, and the maximum input size was *8,192 bytes*. The coverage was found to be *4/8 (50%),* indicating that half of the program's branches were explored.

```
hasib@runner-24:~/fuzzing-lab/basic$ honggfuzz -i input_hf -o output_hf -- ./vulnerable_hf ___FILE___
Persistent signature found in './vulnerable_hf'. Enabling persistent fuzzing mode
Start time:'2026-02-26.16.11.50' bin:'./vulnerable_hf', input:'input_hf', output:'output_hf', persistent:true, stdin:false, mutation_rate:5, timeout:1, max_runs:0,
 threads:1, minimize:false, git_commit:01713bdad7307acea887517f2468b4a9961f9806




----------------------[  0 days 00 hrs 07 mins 23 secs ]--------------------
  Iterations : 6,474 [6.47k]
  Mode [3/3] : Feedback Driven Mode
      Target : ./vulnerable_hf ___FILE___
     Threads : 1, CPUs: 2, CPU%: 100% [50%/CPU]
       Speed : 0/sec [avg: 14]
     Crashes : 275 [unique: 1, blocklist: 0, verified: 0]
    Timeouts : 189 [1 sec]
 Corpus Size : 11, max: 8,192 bytes, init: 1 files
  Cov Update : 0 days 00 hrs 07 mins 07 secs ago
    Coverage : edge: 4/8 [50%] pc: 0 cmp: 193
------------------------------- [ LOGS ] -----------------/ honggfuzz 2.6 /-
[2026-02-26T16:19:12+0300][W][2558087] arch_checkWait():237 Persistent mode: pid=2565500 exited with status: EXITED, exit code: 0
Persistent mode: Launched new persistent pid=2565501
Crash (dup): '/home/hasib/fuzzing-lab/basic/SIGABRT.PC.55555561b82a.STACK.156dd97942.CODE.-6.ADDR.0.INSTR.lea____-0x848(%rbp),%rdx.fuzz' already exists, skipping
[2026-02-26T16:19:13+0300][W][2558087] arch_checkWait():237 Persistent mode: pid=2565501 exited with status: SIGNALED, signal: 6 (Aborted)
Persistent mode: Launched new persistent pid=2565504
[2026-02-26T16:19:13+0300][W][2558087] arch_checkWait():237 Persistent mode: pid=2565504 exited with status: EXITED, exit code: 0
Persistent mode: Launched new persistent pid=2565505
[2026-02-26T16:19:13+0300][W][2558087] arch_checkWait():237 Persistent mode: pid=2565505 exited with status: EXITED, exit code: 0
```

```
------------------------------- [ LOGS ] -----------------/ honggfuzz 2.6 /-
[2026-02-26T16:19:12+0300][W][2558087] arch_checkWait():237 Persistent mode: pid=2565500 exited with status: EXITED, exit code: 0
Persistent mode: Launched new persistent pid=2565501
Crash (dup): '/home/hasib/fuzzing-lab/basic/SIGABRT.PC.55555561b82a.STACK.156dd97942.CODE.-6.ADDR.0.INSTR.lea____-0x848(%rbp),%rdx.fuzz' already exists, skipping
[2026-02-26T16:19:13+0300][W][2558087] arch_checkWait():237 Persistent mode: pid=2565501 exited with status: SIGNALED, signal: 6 (Aborted)
Persistent mode: Launched new persistent pid=2565504
[2026-02-26T16:19:13+0300][W][2558087] arch_checkWait():237 Persistent mode: pid=2565504 exited with status: EXITED, exit code: 0
Persistent mode: Launched new persistent pid=2565505
[2026-02-26T16:19:13+0300][W][2558087] arch_checkWait():237 Persistent mode: pid=2565505 exited with status: EXITED, exit code: 0
Persistent mode: Launched new persistent pid=2565506
[2026-02-26T16:19:13+0300][W][2558087] arch_checkWait():237 Persistent mode: pid=2565506 exited with status: EXITED, exit code: 0
Persistent mode: Launched new persistent pid=2565507
[2026-02-26T16:19:13+0300][W][2558087] arch_checkWait():237 Persistent mode: pid=2565507 exited with status: EXITED, exit code: 0
Persistent mode: Launched new persistent pid=2565508
[2026-02-26T16:19:13+0300][W][2558087] arch_checkWait():237 Persistent mode: pid=2565508 exited with status: EXITED, exit code: 0
Persistent mode: Launched new persistent pid=2565509
Signal 2 (Interrupt) received, terminating
Terminating thread no. #0, left: 0
Summary iterations:6474 time:443 speed:14 crashes_count:275 timeout_count:189 new_units_added:9 slowest_unit_ms:1108 guard_nb:8 branch_coverage_percent:50 peak_rss
_mb:72
```

Then I pressed *Ctrl + C* to stop.

See where and what the crash files are:

```
hasib@runner-24:~/fuzzing-lab/basic$ ls -la output_hf
total 68
drwx------ 2 hasib hasib  4096 Feb 26 16:12 .
drwxrwxr-x 7 hasib hasib 20480 Feb 26 16:19 ..
-rw-r--r-- 1 hasib hasib     5 Feb 26 16:11 0ebc9d2d33d00000438d2d2ccee00000.00000005.honggfuzz.cov
-rw-r--r-- 1 hasib hasib   897 Feb 26 16:12 120cdf85800271bf3e639c9a4ecf02c0.00000381.honggfuzz.cov
-rw-r--r-- 1 hasib hasib    29 Feb 26 16:11 2d4318de8610adb43d20045f6b8ee4c6.0000001d.honggfuzz.cov
-rw-r--r-- 1 hasib hasib  3129 Feb 26 16:11 320f2ef77e4602bdce6e4c40e901058e.00000c39.honggfuzz.cov
-rw-r--r-- 1 hasib hasib    15 Feb 26 16:11 34868513a4e32cd936e02ee0bcb6c802.0000000f.honggfuzz.cov
-rw-r--r-- 1 hasib hasib    69 Feb 26 16:11 41872ab995cad678f76dbfde47d13f79.00000045.honggfuzz.cov
-rw-r--r-- 1 hasib hasib     4 Feb 26 16:11 5c9d2d0b300000007b6d2d2cc0000000.00000004.honggfuzz.cov
-rw-r--r-- 1 hasib hasib     4 Feb 26 16:11 5c9d2d33d00000000438d2d2cc0000000.00000004.honggfuzz.cov
-rw-r--r-- 1 hasib hasib    15 Feb 26 16:11 8662d312af4b012fc8c18b34eeafc189.0000000f.honggfuzz.cov
-rw-r--r-- 1 hasib hasib    41 Feb 26 16:11 e4401143d651d0efdc71a3dd719fc3e7.00000029.honggfuzz.cov
-rw-r--r-- 1 hasib hasib    31 Feb 26 16:11 fae1fcd638d7bef9f63a9e5ddf5e453d.0000001f.honggfuzz.cov
```

Crash replay:

The crash files shown in *output_hf*:

```
hasib@runner-24:~/fuzzing-lab/basic$ ./vulnerable_hf output_hf/120cdf85800271bf3e639c9a4ecf02c0.00000381.honggfuzz.cov
=================================================================
==2565521==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x799d9d700084 at pc 0x5bd6ca09282a bp 0x7fff0d3094d0 sp 0x7fff0d308c88
WRITE of size 898 at 0x799d9d700084 thread T0


^C
```

```
hasib@runner-24:~/fuzzing-lab/basic$ ./vulnerable_hf output_hf/320f2ef77e4602bdce6e4c40e901058e.00000c39.honggfuzz.cov
=================================================================
==2565536==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x77e80cb00084 at pc 0x5a8a5d46082a bp 0x7ffd1e3d8f50 sp 0x7ffd1e3d8708
WRITE of size 359 at 0x77e80cb00084 thread T0
    #0 0x5a8a5d460829 in memcpy (/home/hasib/fuzzing-lab/basic/vulnerable_hf+0xc7829) (BuildId: 4c8cc892ee1b1ba2685d360910ad961651a256df)
    #1 0x5a8a5d49fe20 in vulnerable_function /home/hasib/fuzzing-lab/basic/vulnerable.c:6:1
    #2 0x5a8a5d4a0183 in main /home/hasib/fuzzing-lab/basic/vulnerable.c:20:1
    #3 0x77e80ea2a1c9 in __libc_start_call_main csu/../sysdeps/nptl/libc_start_call_main.h:58:16
    #4 0x77e80ea2a28a in __libc_start_main csu/../csu/libc-start.c:360:3
    #5 0x5a8a5d3c68f4 in _start (/home/hasib/fuzzing-lab/basic/vulnerable_hf+0x2d8f4) (BuildId: 4c8cc892ee1b1ba2685d360910ad961651a256df)

Address 0x77e80cb00084 is located in stack of thread T0 at offset 132 in frame
    #0 0x5a8a5d49fcef in vulnerable_function /home/hasib/fuzzing-lab/basic/vulnerable.c:4

  This frame has 1 object(s):
    [32, 132) 'buffer' (line 5) <== Memory access at offset 132 overflows this variable
HINT: this may be a false positive if your program uses some custom stack unwind mechanism, swapcontext or vfork
      (longjmp and C++ exceptions *are* supported)
SUMMARY: AddressSanitizer: stack-buffer-overflow (/home/hasib/fuzzing-lab/basic/vulnerable_hf+0xc7829) (BuildId: 4c8cc892ee1b1ba2685d360910ad961651a256df) in memcp
y
Shadow bytes around the buggy address:
  0x77e80caffe00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x77e80caffe80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x77e80cafff00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x77e80cafff80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x77e80cb00000: f1 f1 f1 f1 00 00 00 00 00 00 00 00 00 00 00 00
=>0x77e80cb00080:[04]f3 f3 f3 f3 f3 f3 f3 00 00 00 00 00 00 00 00
  0x77e80cb00100: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x77e80cb00180: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x77e80cb00200: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x77e80cb00280: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x77e80cb00300: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

```
Shadow bytes around the buggy address:
  0x77e80caffe00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x77e80caffe80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x77e80cafff00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x77e80cafff80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x77e80cb00000: f1 f1 f1 f1 00 00 00 00 00 00 00 00 00 00 00 00
=>0x77e80cb00080:[04]f3 f3 f3 f3 f3 f3 f3 00 00 00 00 00 00 00 00
  0x77e80cb00100: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x77e80cb00180: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x77e80cb00200: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x77e80cb00280: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x77e80cb00300: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
Shadow byte legend (one shadow byte represents 8 application bytes):
  Addressable:           00
  Partially addressable: 01 02 03 04 05 06 07
  Heap left redzone:       fa
  Freed heap region:       fd
  Stack left redzone:      f1
  Stack mid redzone:       f2
  Stack right redzone:     f3
  Stack after return:      f5
  Stack use after scope:   f8
  Global redzone:          f9
  Global init order:       f6
  Poisoned by user:        f7
  Container overflow:      fc
  Array cookie:            ac
  Intra object redzone:    bb
  ASan internal:           fe
  Left alloca redzone:     ca
  Right alloca redzone:    cb
==2565536==ABORTING
```

Comparison of *Honggfuzz*'s effectiveness with *AFL++* vs *Honggfuzz*:

Analyzing the output of *Honggfuzz* shows that it completed a total of 6,474 *iterations* in a run time of about 7 minutes and worked in *Feedback-Driven Mode*, that is, it mutated the input according to coverage. Although there were a total of *275 crash* detections, there was 1 unique crash, which in our case indicates that the same type of *stack-buffer-overflow* is triggered repeatedly. *189 timeouts* were also found, which shows that some input made the program slow or behave abnormally. During the crash replay, ASAN clearly showed that the *memcpy/copy* operation exceeded the buffer (*size 100*) of *vulnerable_function()* and wrote up to *359 bytes,* resulting in a *stack-buffer-overflow*. This proves that *Honggfuzz* was able to successfully create overflow-triggering input and provided a clean stack trace with sanitizer.

On the other hand, *AFL++* completed *120k executions* in about *11 minutes,* where there were 1097 total crashes but 1 unique saved crash. AFL++ had a much higher execution speed *(≈178 exec/sec)* than *Honggfuzz*, and its UI dashboard showed very detailed coverage, corpus, and crash statistics. In summary, both tools successfully detected the same *buffer overflow vulnerability;* however, *AFL++* provided a higher execution rate and stronger corpus management, while *Honggfuzz* provided clean and fast crash analysis with ASAN-integrated debugging and persistent mode, despite requiring relatively fewer iterations. *Honggfuzz* is relatively lightweight and ASAN-enabled, and shows a fast and clean sanitizer stack trace when a crash occurs, and saves crash artifacts to *output_hf/.* In terms of effectiveness, both are capable of detecting buffer overflow type bugs, but *AFL++* generally provides more structured evidence for coverage-guided mutation and UI/corpus management, while *Honggfuzz* is convenient for fast setup and sanitizer-friendly debugging.

2.

Python Random Fuzzer:

Creating *venv:*

```
hasib@runner-24:~/fuzzing-lab/basic$ python3 -m venv .venv
```

***venv* Activate:**

```
hasib@runner-24:~/fuzzing-lab/basic$ source .venv/bin/activate
```

Script Ready:

Then the following script is inserted into the *py_random_fuzzer.py* file.

```
(.venv) hasib@runner-24:~/fuzzing-lab/basic$ nano py_random_fuzzer.py
```

```
 GNU nano 7.2                                                py_random_fuzzer.py
import os
import random
import subprocess
import time

TARGET = "./vulnerable"
OUTDIR = "py_fuzz_output"
MAX_TESTS = 5000

os.makedirs(OUTDIR, exist_ok=True)

def main():
    crashes = 0
    start = time.time()

    for i in range(1, MAX_TESTS + 1):
        length = random.randint(0, 4000)
        data = os.urandom(length)

        fname = os.path.join(OUTDIR, f"case_{i}.bin")
        with open(fname, "wb") as f:
            f.write(data)

        p = subprocess.run([TARGET, fname],
                           stdout=subprocess.DEVNULL,
                           stderr=subprocess.DEVNULL)
```

```
        if p.returncode != 0:
            crashes += 1
            crash_name = os.path.join(OUTDIR, f"CRASH_{i}_ret{p.returncode}.bin")
            os.rename(fname, crash_name)
            print(f"[!] Crash saved: {crash_name} (returncode={p.returncode})")
        else:
            os.remove(fname)

        if i % 200 == 0:
            print(f"[*] tests={i}, crashes={crashes}, elapsed={time.time()-start:.1f}s")

    print(f"\nDone. total_tests={MAX_TESTS}, crashes={crashes}")
    print(f"Crash samples saved in: {OUTDIR}")

if __name__ == "__main__":
    main()

^G Help        ^O Write Out   ^W Where Is   ^K Cut         ^T Execute     ^C Location    M-U Undo
^X Exit        ^R Read File   ^\ Replace    ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo
```

Run:

A total of *5000 tests* were successfully run here (*Done. total_tests=5000*) and the entire process was completed in about *18.6 seconds*.

```
(.venv) hasib@runner-24:~/fuzzing-lab/basic$ python py_random_fuzzer.py
[*] tests=200, crashes=0, elapsed=0.8s
[*] tests=400, crashes=0, elapsed=1.5s
[*] tests=600, crashes=0, elapsed=2.3s
[*] tests=800, crashes=0, elapsed=3.0s
[*] tests=1000, crashes=0, elapsed=3.8s
[*] tests=1200, crashes=0, elapsed=4.5s
[*] tests=1400, crashes=0, elapsed=5.2s
[*] tests=1600, crashes=0, elapsed=6.0s
[*] tests=1800, crashes=0, elapsed=6.7s
[*] tests=2000, crashes=0, elapsed=7.5s
[*] tests=2200, crashes=0, elapsed=8.2s
[*] tests=2400, crashes=0, elapsed=8.9s
[*] tests=2600, crashes=0, elapsed=9.7s
[*] tests=2800, crashes=0, elapsed=10.4s
[*] tests=3000, crashes=0, elapsed=11.1s
[*] tests=3200, crashes=0, elapsed=11.9s
[*] tests=3400, crashes=0, elapsed=12.6s
[*] tests=3600, crashes=0, elapsed=13.4s
[*] tests=3800, crashes=0, elapsed=14.1s
[*] tests=4000, crashes=0, elapsed=14.8s
[*] tests=4200, crashes=0, elapsed=15.6s
[*] tests=4400, crashes=0, elapsed=16.3s
[*] tests=4600, crashes=0, elapsed=17.1s
[*] tests=4800, crashes=0, elapsed=17.9s
[*] tests=5000, crashes=0, elapsed=18.6s

Done. total_tests=5000, crashes=0
Crash samples saved in: py_fuzz_output
```

Crash replay:

If you check the *py_fuzz_output* folder, you will see that there is no *CRASH* file there. This means that this random *fuzzer* could not trigger a buffer overflow. Because, it uses completely blind random input, no coverage feedback or targeted mutation, as is the case with *AFL++* or *Honggfuzz*.

```
(.venv) hasib@runner-24:~/fuzzing-lab/basic$ ls -la py_fuzz_output | head
total 24
drwxrwxr-x 2 hasib hasib  4096 Feb 26 16:31 .
drwxrwxr-x 9 hasib hasib 20480 Feb 26 16:30 ..
```

References:

[1] B. P. Miller, L. Fredriksen, and B. So, "An Empirical Study of the Reliability of UNIX Utilities," *Commun. ACM*, vol. 33, no. 12, pp. 32–44, Dec. 1990, doi: *10.1145/96267.9627*. Available: https://dl.acm.org/doi/10.1145/96267.96279

[2] M. Zalewski, "american fuzzy lop (AFL)," *coredump.cx*. Available: https://lcamtuf.coredump.cx/afl/

[3] AFLplusplus Project, "AFLplusplus/AFLplusplus (GitHub repository & docs)," *GitHub*. Available: https://github.com/AFLplusplus/AFLplusplus

[4] M. Böhme, V.-T. Pham, and A. Roychoudhury, "Coverage-based Greybox Fuzzing as Markov Chain," in *Proc. ACM CCS*, 2016, pp. 1032-1043, doi: *10.1145/2976749.2978428.* Available: https://dblp.org/rec/conf/ccs/BohmePR16.html

[5] LLVM Project, "libFuzzer - a library for coverage-guided fuzz testing," *LLVM Documentation*. Available: https://llvm.org/docs/LibFuzzer.html

[6] LLVM Project, "AddressSanitizer," *Clang Documentation*. Available: https://clang.llvm.org/docs/AddressSanitizer.html

[7] MITRE, "CWE-120: Buffer Copy without Checking Size of Input ('Classic Buffer Overflow')," *CWE*. Available: https://cwe.mitre.org/data/definitions/120.html